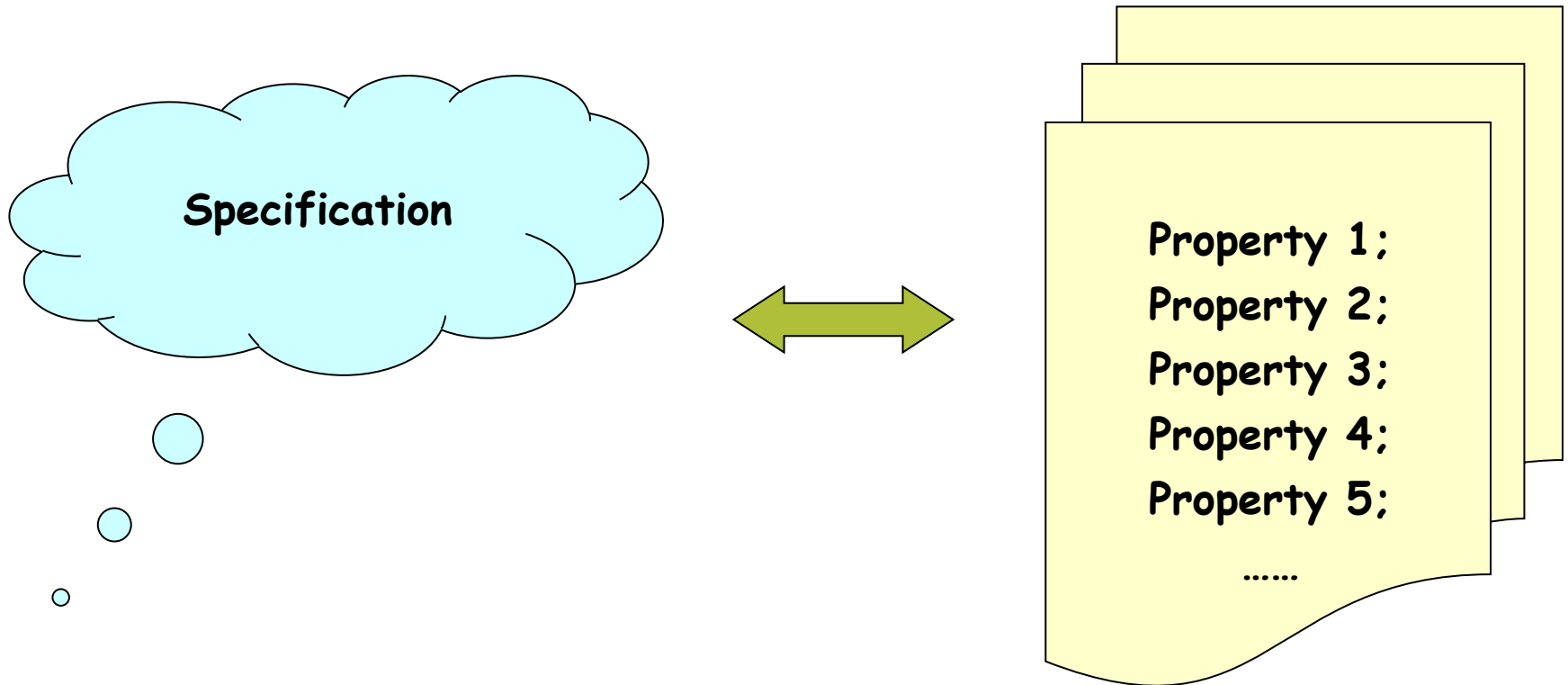




Property Characterization



Specification = Set of Properties



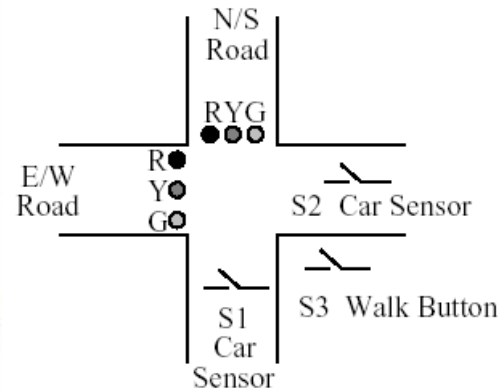
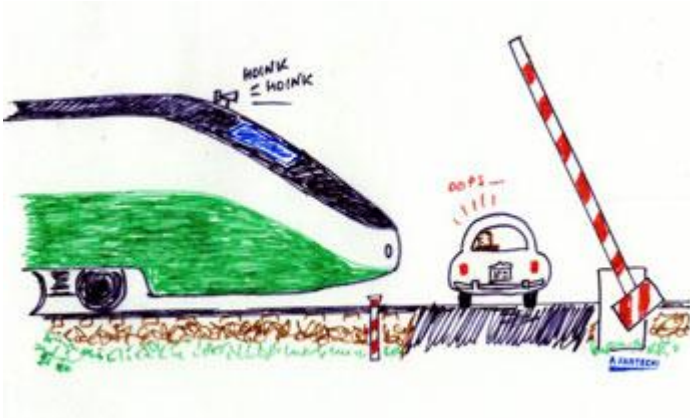
Characterizing "Behaviors"

- An **assertion** is a statement about a specific functional characteristic or property that is expected to hold for a design.
- We use:
 - Assertions (overloaded in SV)
 - Properties

1. Safety Properties

- **Definition**
 - Safety properties state that “something bad never happens.”
- **Invariants:** they assert that bad states are never reachable.

Safety (Invariants): Basic Requirements



- A safety property states that **some bad thing never happens**.
- Safety properties represent requirements that should be **continuously maintained by the system**.
- Basic requirements.

Safety Property

- It guarantees that the system **never enters an incorrect or undesirable state.**
- A property P is a **safety property** if **every violation of P has a finite witness**
 - i.e., a finite sequence of events leading to an error.
- If an error occurs, it is observable **within a finite execution trace.**
- A violation can be detected by a finite prefix of an execution.

Safety Properties in LTL

■ Mutual Exclusion

- Two processes are never in the critical section at the same time.
- $G \neg(cs1 \wedge cs2)$

■ No Invalid State

- The system never enters an illegal configuration.
- $G \neg error$

■ Protocol Consistency

- A grant can only be issued if there is a pending request.
- $G (grant \rightarrow request)$

Safety Properties in CTL

■ Request-Grant Consistency

- A grant is never issued unless a corresponding request exists.
- $AG (\text{grant} \rightarrow \text{request})$

■ No Simultaneous Read-Write (Memory Safety)

- A memory location is never read and written at the same time.
- $AG \neg(\text{read} \wedge \text{write})$

■ Error State Is Unreachable

- The system never enters an error state.
- $AG \neg \text{error}$

Applications of Safety Properties

- **Hardware Design:** Ensuring correct synchronization in circuits.
- **Software Verification:** Avoiding buffer overflows and race conditions.
- **Security Protocols:** Preventing unauthorized access or data leaks.
- **Autonomous Systems:** Ensuring safe operation of self-driving cars and robots.

- **Mutual Exclusion:** Two processes should never be in the critical section simultaneously.
- **No Data Corruption:** A variable should never hold an invalid value.
- **Packet Integrity:** A transmitted message should not be altered unexpectedly.
- **Memory Safety:** A program should never access memory out of bounds.

Verification of Safety Properties

- **Model Checking:**
 - Ensures that no execution leads to an unsafe state
 - (e.g., using **Temporal Logic** such as **LTL**, **CTL**).
- **Theorem Proving:**
 - Uses logical proofs to establish correctness
 - (e.g., **Hoare Logic**, **Isabelle**, **Coq**).
- **Runtime Verification:**
 - Monitors system execution to check for violations.

Summary: Safety Properties

- Properties whose minimal error traces are finite
- A safety property corresponds to
 - partial correction which does not ensure termination,
 - but only that all terminating computations produce correct results.
- Key Characteristics
 - Prefix-closed
 - Counterexample is finite
 - Often written using G (globally)
- A **safety property** is a fundamental correctness requirement in formal verification, ensuring that the system never reaches an undesirable state.
- It is crucial in designing **reliable, secure, and error-free** software and hardware systems.

2. Liveness Properties

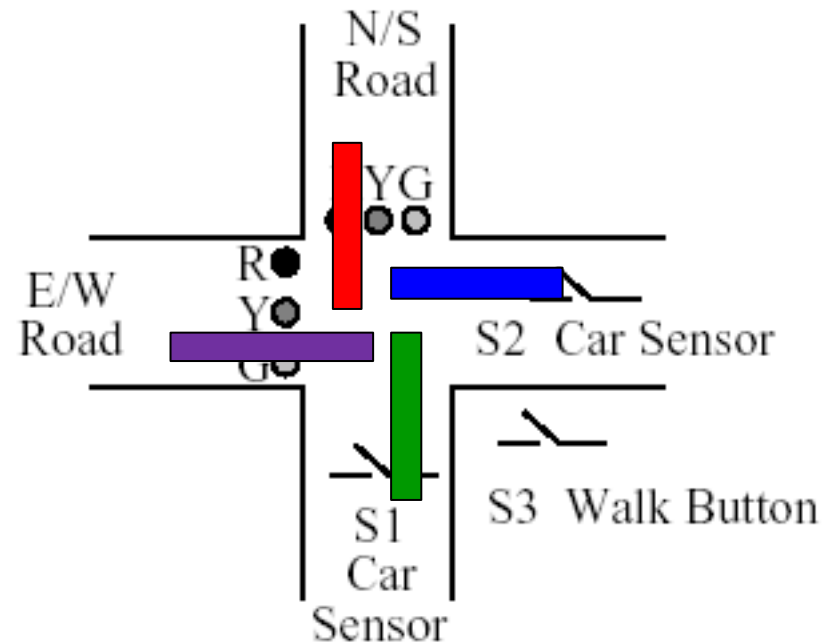
- **Definition**

- Liveness properties state that “something good eventually happens.”

- **Formal intuition**

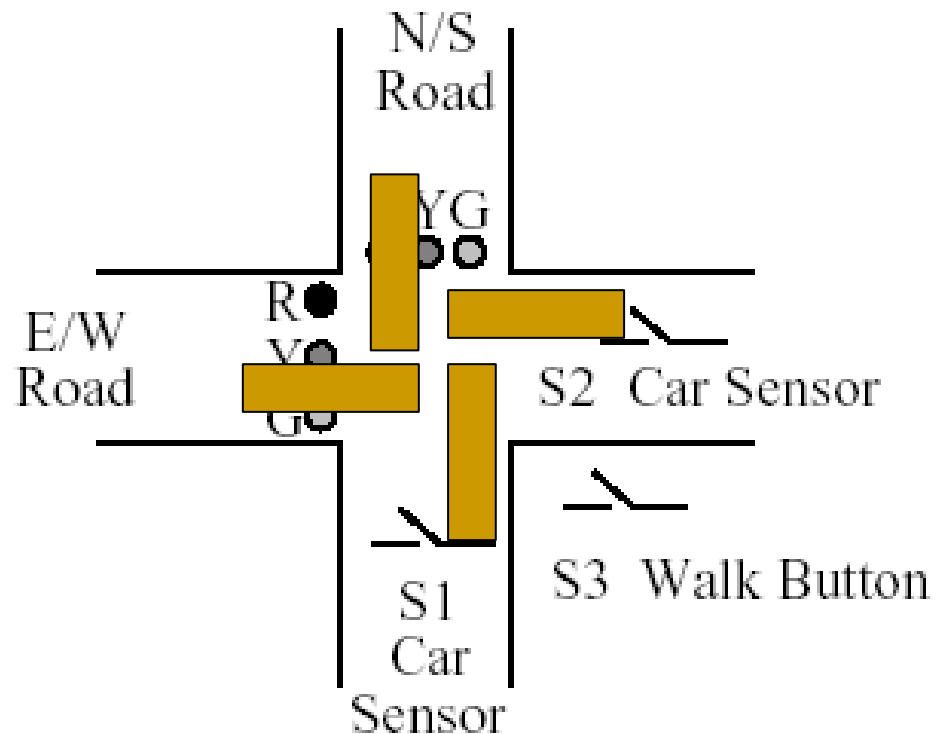
- No finite prefix can prove failure — the system *might* still succeed later.
- Violations can only be seen in **infinite executions**.

Negation of Infinite Jams



- A = Bad Event = "Loop (Jam) goes forever (infinitely)"
- $\text{NOT}(A) = ?$

Liveness = NOT (Infinite Loop)



- A = "Bad infinite loop goes forever" = deadlock = infinite loop
- $\text{NOT}(A)$ = ? "something good eventually happens."

Liveness Property

- A **liveness property** ensures that "something good eventually happens" in a system.
- It guarantees that the system does not get stuck in an undesirable state **indefinitely** and that progress is made.
- Liveness properties correspond to total correctness which guarantees termination for program verification.
- A liveness property states that **some good thing eventually happens**.
 - **But we don't know when !**
- Liveness properties represent requirements that
 - need not hold **continuously**,
 - but those **eventual (or repeated) realization** must be ensured.

Liveness Examples in LTL

- **Eventual access in a mutual exclusion system**
 - If a process requests the critical section, it will eventually enter it. Let $request_1$ = process 1 requests, in_cs_1 = process 1 in critical section.
- **LTL formula:**
 - $G(request_1 \rightarrow F in_cs_1)$
- **Explanation:**
 - G = globally (always)
 - F = eventually (at some future point)
 - Meaning: whenever P1 requests, it **will eventually enter the critical section**.
 - Nontrivial because it prevents starvation in mutual exclusion systems.

Liveness Examples in LTL

- **Eventual response to a request**
 - In a request-response system, every request must eventually get a response. Let request and response be atomic propositions.
- **LTL formula:**
 - $G(\text{request} \rightarrow F\text{response})$
- **Explanation:**
 - Guarantees **every request is eventually answered**.
 - Standard liveness property in communication protocols.

Liveness Examples in LTL

- **Recurring safety (infinitely often)**
 - A system repeatedly reaches a “safe” state, denoted by *safe*.
- **LTL formula:**
 - $GFsafe$
- **Explanation:**
 - $F\ safe$ = eventually safe
 - $G\ F\ safe$ = infinitely often, the system will return to a safe state.
 - Nontrivial because it ensures **recurrent recovery or progress**.

Liveness Examples in LTL

- **Eventual task completion**
 - Every pending task eventually finishes. Let $\text{task_pending} = \text{task exists}$, $\text{task_done} = \text{task completed}$.
- **LTL formula:**
 - $G(\text{task_pending} \rightarrow F\text{task_done})$
- **Explanation:**
 - Ensures tasks do not remain forever pending.
 - Nontrivial in concurrent or fair scheduling systems.

Liveness Examples in CTL

- **Eventual success on some path**
 - **Property:** There exists a path where a process eventually enters the critical section.
- **CTL formula:**
 - $EFin_cs_1$
- **Explanation:**
 - $EF \varphi$ = there exists **some path** where eventually φ happens.
 - Nontrivial: asserts **at least one possible execution** reaches the critical section.

Liveness Examples in CTL

- **Possible response to a request**
 - **Property:** There exists a path where a request eventually gets a response.
- **CTL formula:**
 - $AG(request \rightarrow EFresponse)$
- **Explanation:**
 - Globally, whenever a request occurs, **there exists a path** where the response eventually happens.
 - Nontrivial: weaker than AF (all paths) but still meaningful—it asserts **possible fulfillment**.

Liveness Examples in CTL

- **Possible recovery (eventual safe state)**
 - **Property:** From any state, there exists a path leading to a safe state.
- **CTL formula:**
 - $AGEFsafe$
- **Explanation:**
 - EF safe = there exists a path eventually reaching safe.
 - AG EF safe = from **every reachable state**, it is **possible** to reach a safe state.
 - Nontrivial: guarantees that recovery is possible even if not guaranteed on all paths.

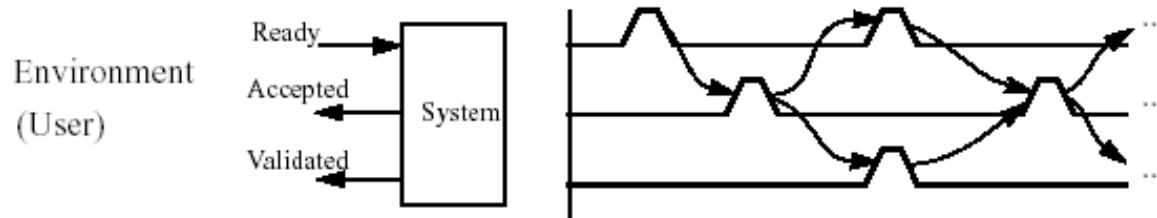
Liveness Examples in CTL

- **Possible task completion**
 - **Property:** Every pending task could eventually be completed along some path.
- **CTL formula:**
 - $AG(task_pending \rightarrow EFtask_done)$
- **Explanation:**
 - Whenever a task is pending, there exists **some execution** where it eventually finishes.
 - Nontrivial: weaker than universal liveness (AF), but still asserts **potential progress**.

Summary: Liveness

- Liveness properties ensure **progress and responsiveness** in systems.
- They are crucial in **concurrent computing, networking, and real-time systems** to avoid issues like deadlocks and starvation.
- Formal methods like **LTL, CTL, and model checking** are used to verify these properties and ensure system correctness.

Examples



Example: A simple interface protocol, pulses one clock period wide

- Safety property:
 - $\forall t. (Validated(t) \rightarrow \neg Validated(t+1))$
 - $G (Validated \rightarrow X \neg Validated)$
- Liveness:
 - $\forall t. (Ready(t) \rightarrow \exists (t' \geq t+1). Accepted(t'))$
 - $G (Ready \rightarrow F Accepted)$

Safety vs Liveness

- **Alpern-Schneider Theorem**

- *Every property can be decomposed into a safety part and a liveness part.*

- **Approximating Liveness with Safety**

- The key technique is **bounded approximation**.

- Liveness says:

- "Grant must eventually happen (no bound)."

- Safety approximations say:

- "Grant must happen within k steps."

- As k increases, the approximation becomes tighter.

- If the system satisfies all such bounded safety properties for every k , then the original liveness property holds.

Safety = G & Liveness= F, U

Property Type	Definition	Example
Safety	"Something bad never happens"	No two processes enter the critical section simultaneously.
Liveness	"Something good eventually happens"	A process waiting for a resource will eventually get access.

- Safety properties are violated by a finite counterexample.
- Liveness properties require infinite execution to check for violations.

3. Fairness

- Fairness relates the **enablement** of an action (whether it *can* happen) to the **execution** of that action (whether it *actually* happens).
- Weak Fairness
 - If *A* is eventually enabled forever, it must be executed infinitely often.
- Strong Fairness
 - If *A* is enabled infinitely often, it must be executed infinitely often.

Finitely Often $\rightarrow$ Infinitely Often

- Consider an event (or state) p along an infinite run:

- **Finitely often**

- ☐ p happens only a finite number of times, then stops forever.



- **Infinitely often**

- ☐ p keeps happening again and again, never stopping.



Why “infinitely often” matters

- Fairness is fundamentally about “infinitely often” behavior.
- Unfair behavior looks like this:
 - A process is ready **again and again** But it is **never chosen** Forever
- Fairness asks:
 - “If something keeps being possible infinitely often, shouldn’t it eventually happen?”
 - That is *precisely* an “infinitely often” question.

Fairness Properties

- “Infinitely often” properties
 - $G F p$ means that p occurs infinitely often (“always eventually p ”).
 - $G F p = \neg F G \neg p$ = “a point never reached where p is forever false”.
- Example:
$$G F \text{ send } (x) \Rightarrow G F \text{ receive } (x)$$
 - “if msg x is sent infinitely often, it is received infinitely often”
- This is an example of a “fairness” property.

Why fairness is NOT just eventuality

- **Eventuality (liveness):**
 - $Fgrant$
 - "Grant happens at least once."
- **Fairness:**
 - $GFreque\!st \rightarrow GFgrant$
 - "If requests keep happening forever, grants must also keep happening."
- Fairness is **stronger and more structural**.

Fairness as a Restriction on Paths

- In temporal logic:
 - Liveness is evaluated over paths
 - Fairness restricts which paths are considered
- So:
 - Liveness = “good things eventually happen”
 - Fairness = “we only consider reasonable executions”
- Mathematically:
 - Liveness holds **under fair paths**, not necessarily under *all* paths.

Fairness in LTL

- This is why fairness is natural in LTL
- “On all paths, p occurs infinitely often”
- LTL:
 - GFp
- In LTL
 - LTL semantics typically assume **fair execution** implicitly, or fairness is added explicitly:
 - $G(\text{request} \rightarrow F\text{grant})$ under fairness

Fairness in CTL

- CTL itself cannot express fairness, so it is given externally.
- This is not expressible in CTL.
 - $AF\ p$ only means "at least once"
 - $AG\ AF\ p$ does **not** mean "infinitely often on each path"
 - $AG\ AF\ p$ means:
 - From every reachable state, there exists a future occurrence of p

Fairness in CTL/LTL

- Liveness fails without fairness because CTL considers unfair infinite paths;
- Fairness constrains CTL by eliminating paths where enabled actions are postponed forever.
- **In CTL**
 - Liveness often needs fairness constraints to validate formulas like:
 - $AG(request \rightarrow AFgrant)$
 - Fairness removes paths where grant is *forever postponed*.
- Typical fairness condition:
 - $fair_1 = GFenabled_1$
- Meaning:
 - If `enabled_1` is true infinitely often, then transitions of P1 must occur infinitely often
- Model checkers (e.g., NuSMV) use:
 - FAIRNESS `enabled_1`

Fairness Constraints for Verification

- A fairness property is a constraint used to rule out unrealistic executions in which a process or event is indefinitely neglected.
- Without fairness assumptions, a model checker may identify unrealistic counterexamples,
 - such as a processor waiting forever because the scheduler continually ignores it;
- Without fairness, it is often impossible to prove liveness properties
 - for example, in a system with two processes, one cannot guarantee that Process B will eventually execute unless the scheduler is assumed to be fair and not run Process A indefinitely.

References

- E. A. Emerson. Temporal Logics in Handbook of Theoretical Computer Science. Elsevier Science Publishers B.V., 1990
- Z. Manna, A. Pnueli. The Temporal Logic of Rective and Concurrent Systems. Springer-Verlag, 1991.
- A. P. Sistla, E. M. Clarke. The complexity of propositional linear temporal logics. JACM, 32(3), 1985, pp. 733-749.
- E. A. Emerson, E.M. Clarke. "Sometimes" and "Not Never" revisited: on branching versus linear time temporal logic. JACM, 33(1), 1986, pp. 151-178.
- Principles of Model Checking by Christel Baier and Joost-Pieter Katoen. The MIT Press; 2008.
- Model checking, by E. M. Clarke, etc. The MIT Press; second edition, 2018.
- M. Huth, M. Ryan, Logic in Computer Science: Modeling and Reasoning about Systems, Cambridge University Press; second edition, 2004.